

REPORTES DE LABORATORIO: ESTRUCTURAS DE DATOS

SEMANA 12: TABLAS HASH Y DICCIONARIOS

Institución: Unidad Académica de Economía

Carrera: Sistemas Computacionales

Materia: Estructura de Datos Básica

Profesor: Eligardo Cruz Sánchez

Alumno: Gil Alexander Ramirez

Checkpoint de Comprensión (Evaluación Grupal)

C1. Manejo de Colisiones en Tablas Hash

Una colisión pasa cuando dos llaves distintas terminan dando exactamente el mismo índice con la función hash. Ojo, no significa que el código esté mal o sea un error de programación; es algo totalmente normal y esperado en las tablas hash porque casi siempre tenemos muchísimas más llaves posibles que espacios (buckets) disponibles en memoria. Para solucionar esto y que no se sobrescriban los datos, se usan estrategias como el Encadenamiento (chaining) o el Direcccionamiento abierto (open addressing).

C2. Cálculo y Análisis del Factor de Carga

El factor de carga lo sacamos dividiendo los elementos guardados entre el total de buckets disponibles:

Factor de carga = $10,000 / 15,000 = 0.6667$ (aproximadamente 0.67).

Este número está en una zona súper segura para el rendimiento. La mayoría de las implementaciones corren de maravilla siempre que este factor se mantenga por debajo de 0.70 o 0.75. Esto nos asegura que quedan bastantes espacios libres y las colisiones no se van a descontrolar.

C3. Acceso Directo vs Método .get()

Si usamos `d['MX-999']` directamente y la llave no existe, el programa se va a romper por completo lanzando un error `KeyError`. En cambio, si usamos `d.get('MX-999')` el código es más seguro porque no truena; simplemente te regresa `None` o un mensaje por defecto si se lo pones (como `d.get('MX-999', 'No encontrado')`).

¿Cuándo preferiría el acceso directo? Cuando esté 100% seguro de que el dato debe existir y, si llega a faltar, signifique un error de lógica grave en el sistema que necesito cachar de inmediato.

C4. Limitaciones del Diccionario para Ordenamiento y Rankings

No, un diccionario no sirve para sacar de forma eficiente cosas como el 'top 5 de películas mejor calificadas'. Las tablas hash se diseñaron específicamente para buscar, meter y borrar datos en un tiempo promedio rapidísimo de $O(1)$. Pero para armar un top 5, a fuerza necesitas revisar todos los elementos y ordenarlos por calificación, lo que nos lleva a una complejidad de $O(n \log n)$. Si ocupamos hacer rankings seguidos, lo ideal sería combinar el diccionario con una lista ordenada, un heap o meter los datos en una base de datos indexada.

Reflexión de Aprendizaje (Conceptos)

Aquí nos cayó el veinte de que los diccionarios no guardan las cosas ordenadas como las listas, sino que usan una función matemática (hash) para mandar cada elemento a una posición. También entendimos que las colisiones son parte del juego y no errores de sintaxis. Antes pensábamos que el dict servía para cualquier tipo de búsqueda rápida, pero ya vimos que si queremos armar 'tops' o rankings nos quedamos cortos y ocupamos estructuras extra.

Código Fuente Modificado (streamMX_hash_s12.py)

```
# =====
# RETO 7: Probando los 4 patrones con diccionarios en StreamMX
# =====
# Notas que aprendimos en el checkpoint:
# - C1: Las colisiones son normales cuando dos llaves dan el mismo hash.
# - C2: Nuestro factor de carga es de 0.67, una zona bastante segura.
# - C3: d[llave] truena con KeyError si falta; d.get() es el camino seguro.
# - C4: Búsquedas en  $O(1)$ , pero para ordenar ocupamos  $O(n \log n)$ .
# =====

import time
import random

# CATÁLOGO PRINCIPAL DE STREAMMX
catalogo = {
    "MX-001": {
        "titulo": "Interestelar",
        "genero": "Ciencia Ficción",
        "puntaje": 9.2,
    },
    "MX-002": {
        "titulo": "El Origen",
        "genero": "Ciencia Ficción",
        "puntaje": 8.8,
    },
    "MX-003": {
        "titulo": "Toy Story",
        "genero": "Animación",
```

```

        "puntaje": 8.1,
    },
    "MX-004": {
        "titulo": "Shrek",
        "genero": "Animacion",
        "puntaje": 7.9,
    },
    "MX-005": {
        "titulo": "Titanic",
        "genero": "Drama",
        "puntaje": 7.8,
    },
    "MX-006": {
        "titulo": "Batman",
        "genero": "Accion",
        "puntaje": 8.5,
    },
    "MX-007": {
        "titulo": "Avengers",
        "genero": "Accion",
        "puntaje": 7.3,
    },
    "MX-008": {
        "titulo": "Coco",
        "genero": "Animacion",
        "puntaje": 9.0,
    },
}

# --- PATRÓN 1: BÚSQUEDA Y AGREGADO DIRECTO ---
def buscar_pelicula(id_pelicula):
    return catalogo.get(id_pelicula)

def agregar_pelicula(id_pelicula, datos):
    catalogo[id_pelicula] = datos

# --- PATRÓN 2: CONTAR FRECUENCIAS ---
# NOTA DE DISEÑO: Usar get() nos ahorra líneas de código y queda
# mucho más limpio para inicializar y sumar en un solo paso.
def contar_por_genero():
    conteos = {}
    for pelicula in catalogo.values():
        genero = pelicula["genero"]
        conteos[genero] = conteos.get(genero, 0) + 1
    return conteos

# --- PATRÓN 3: AGRUPAR POR RANGOS DE PUNTAJE ---
def agrupar_por_rango():
    grupos = {
        "popular": [],
        "destacado": [],
        "premium": [],
    }

```

```

    }
    for pelicula in catalogo.values():
        puntaje = pelicula["puntaje"]
        if puntaje >= 7.0 and puntaje < 8.0:
            grupos["popular"].append(pelicula["titulo"])
        elif puntaje >= 8.0 and puntaje < 9.0:
            grupos["destacado"].append(pelicula["titulo"])
        elif puntaje >= 9.0 and puntaje <= 10.0:
            grupos["premium"].append(pelicula["titulo"])
    return grupos

def peliculas_en_rango(rango):
    grupos = agrupar_por_rango()
    return grupos.get(rango, [])

# --- PATRÓN 4: QUITAR DUPLICADOS (DEDUPLICACIÓN) ---
def procesar_lote(lista_ids):
    unicos = {}
    for pelicula_id in lista_ids:
        unicos[pelicula_id] = True
    return list(unicos.keys())

# --- PATRÓN 5: MECANISMO DE CACHÉ / MEMOIZACIÓN ---
cache_recomendaciones = {}

def _generar_recomendaciones(genero):
    time.sleep(1) # Simulamos una consulta pesada que tarda un segundo
    recomendaciones = []
    for pelicula in catalogo.values():
        if pelicula["genero"] == genero:
            recomendaciones.append(pelicula["titulo"])
    return recomendaciones

def obtener_recomendaciones(genero):
    if genero in cache_recomendaciones:
        return cache_recomendaciones[genero]
    resultado = _generar_recomendaciones(genero)
    cache_recomendaciones[genero] = resultado
    return resultado

# --- BENCHMARK COMPARATIVO ---
def benchmark_comparativo(n):
    lista = list(range(n))
    diccionario = {i: i for i in range(n)}
    objetivo = random.randint(0, n - 1)

    inicio_lista = time.perf_counter()
    objetivo in lista
    fin_lista = time.perf_counter()
    tiempo_lista = fin_lista - inicio_lista

    inicio_dict = time.perf_counter()

```

```

    objetivo in diccionario
    fin_dict = time.perf_counter()
    tiempo_dict = fin_dict - inicio_dict

    speedup = tiempo_lista / tiempo_dict
    return {
        "n": n,
        "lista": tiempo_lista,
        "dict": tiempo_dict,
        "speedup": speedup,
    }

# EXPLICACIÓN DEL RENDIMIENTO:
# El diccionario gana por goleada porque calcula la posición exacta en
# memoria
# usando hashing, logrando un tiempo promedio de O(1).
# La lista tiene que barrer elemento por elemento (O(n)), por lo que
# entre más grande sea n, más fea se pone la diferencia de tiempos.
# =====

if __name__ == "__main__":
    print("\n===== PATRÓN 1 =====")
    nueva_pelicula = {
        "titulo": "Dune Parte 2",
        "genero": "Ciencia Ficción",
        "puntaje": 8.9,
    }
    agregar_pelicula("MX-013", nueva_pelicula)
    print(buscar_pelicula("MX-013"))

    print("\n===== PATRÓN 2 =====")
    print(contar_por_genero())

    print("\n===== PATRÓN 3 =====")
    grupos = agrupar_por_rango()
    print("Popular:", grupos["popular"])
    print("Destacado:", grupos["destacado"])
    print("Premium:", grupos["premium"])

    print("\n===== PATRÓN 4 =====")
    lote = ["MX-001", "MX-001", "MX-002", "MX-003", "MX-003", "MX-004"]
    print("Original:", lote)
    print("Sin duplicados:", procesar_lote(lote))

    print("\n===== PATRÓN 5 =====")
    inicio_1 = time.perf_counter()
    print(obtener_recomendaciones("Animacion"))
    print("Tiempo primer llamado:", time.perf_counter() - inicio_1)

    inicio_2 = time.perf_counter()
    print(obtener_recomendaciones("Animacion"))
    print("Tiempo segundo llamado (caché):", time.perf_counter() -

```

```
inicio_2)

print("\n===== BENCHMARK =====")
for tam in [10000, 100000, 1000000]:
    print(benchmark_comparativo(tam))
```

Bitácora de Revisión y Conclusiones reales

Sobre el Benchmark y la realidad del $O(1)$:

Al principio la IA nos hizo ver que teníamos una idea muy teórica de $O(1)$. Pensábamos que significaba 'tiempo cero' o instantáneo en cualquier lado. Ya viendo los números reales en la computadora, te das cuenta de que influyen cosas como el uso de memoria de Python o el caché de la máquina. Aun así, la ventaja del diccionario es brutal cuando los datos crecen: pasa de ser 120 veces más veloz con 10,000 elementos a casi 7,000 veces más rápido cuando metemos un millón. Queda clarísimo que para búsquedas exactas por ID o corrección de duplicados, el dict es el rey.

Reporte de Pruebas (VALIDA):

- T1 y T2 pasaron limpios al inicio porque la lógica de buscar por clave y contar frecuencias con diccionarios es muy directa.
- T3 y T4 nos dieron lata. Tuvimos un error de frontera (los valores límite como 8.0 o 9.0 se guardaban en los grupos equivocados) y tuvimos que ajustar los operadores lógicos.
- T5 jaló bien usando la propiedad de que los diccionarios no aceptan llaves repetidas.
- T6 falló al principio porque el caché no estaba guardando bien el resultado, así que seguía ejecutando la función lenta. Una vez corregido, el segundo llamado bajó a casi 0 segundos, dándonos un speedup de más de 100x.

Mirada hacia la Semana 13:

Viendo que el diccionario sufre cuando queremos cosas ordenadas o búsquedas por rangos de valores, ya nos adelantaron que en la Semana 13 veremos Árboles Binarios de Búsqueda (BST). Estas estructuras sí mantienen una jerarquía ordenada basada en 'menor y mayor', lo que nos va a permitir solucionar búsquedas en rangos de puntajes de forma mucho más inteligente que andar recorriendo todo desde cero.